React — Zero to Master

23 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (7 topics)

01 Introduction and Setup

02 JSX

03 Components and Props

04 State with useState

05 Events and Conditional Rendering

06 Lists and Keys

07 Forms and Controlled Inputs

Intermediate (6 topics)

08 useEffect and Side Effects

09 Data Fetching

10 useRef, useMemo, useCallback

11 Context API

12 React Router

13 Custom Hooks

Advanced (6 topics)

14 useReducer and State Patterns

15 Global State: Redux Toolkit, Zustand

16 Performance Optimisation

17 Forms at Scale

18 Server State with TanStack Query

19 Testing React

Master (4 topics)

20	React 19 Features
21	Project Architecture and Best Practices
22	Real Project: E-commerce Front End
23	Deployment and Interview Prep

Beginner Level

01. Introduction and Setup

React is a JavaScript library for building UIs from components. Create an app with Vite.

```
npm create vite@latest my-app -- --template react-ts
cd my-app && npm install && npm run dev
```

02. JSX

JSX looks like HTML inside JavaScript. Use `{}` for expressions, `className` instead of `class`, and return one root element.

```
const App = () => <h1 className="title">Hello {name}</h1>;
```

03. Components and Props

A component is a function returning JSX. Props pass data from parent to child and are read-only.

```
function Card({ title }: { title: string }) { return <h2>{title}</h2>; }
<Card title="React" />
```

04. State with useState

State is data that changes over time; updating it re-renders the component.

```
const [count, setCount] = useState(0);
<button onClick={() => setCount(c => c + 1)}>{count}</button>
```

05. Events and Conditional Rendering

Handle events with camelCase props; render conditionally with `&&`, ternary or early return.

```
{loggedIn ? <Dashboard /> : <Login />}
{items.length > 0 && <List items={items} />}
```

06. Lists and Keys

Render arrays with `map` and give each item a stable unique `key`.

```
{todos.map(t => <li key={t.id}>{t.title}</li>)}
```

07. Forms and Controlled Inputs

A controlled input's value comes from state and changes through `onChange`.

```
<input value={name} onChange={e => setName(e.target.value)} />
```

Intermediate Level

08. useEffect and Side Effects

`useEffect` runs after render for fetching, subscriptions and timers; return a cleanup function; the dependency array controls when it re-runs.

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id);
}, []);
```

09. Data Fetching

Fetch in effects (or use TanStack Query) and track loading and error states; handle race conditions with cleanup/abort.

```
useEffect(() => {
  fetch('/api/users').then(r => r.json()).then(setUsers);
}, []);
```

10. useRef, useMemo, useCallback

`useRef` holds mutable values/DOM nodes without re-rendering; `useMemo` and `useCallback` cache values and functions between renders.

```
const inputRef = useRef<HTMLInputElement>(null);
const total = useMemo(() => sum(items), [items]);
```

11. Context API

Share data (theme, user) across the tree without prop drilling.

```
const ThemeContext = createContext('light');
const theme = useContext(ThemeContext);
```

12. React Router

Client-side routing with `createBrowserRouter`, `Link`, nested routes, params and loaders.

```
<Route path="/users/:id" element={<User />} />
const { id } = useParams();
```

13. Custom Hooks

Extract reusable stateful logic into functions starting with `use`.

```
function useToggle(initial = false) {
  const [on, set] = useState(initial);
  return [on, () => set(v => !v)] as const;
}
```

Advanced Level

14. useReducer and State Patterns

Use `useReducer` for complex state transitions; lift state up and keep state minimal and derived.

```
const [state, dispatch] = useReducer(reducer, { count: 0 });
dispatch({ type: 'inc' });
```

15. Global State: Redux Toolkit, Zustand

For app-wide state use Redux Toolkit (slices, thunks) or lightweight Zustand stores.

```
const useStore = create<{ n: number; inc: () => void }>(set => ({ n: 0, inc: () => set(s => ({
n: s.n + 1 })) }));
```

16. Performance Optimisation

`React.memo`, code splitting with `lazy` / `Suspense`, list virtualisation, avoiding unnecessary renders; measure with React DevTools Profiler.

```
const Chart = lazy(() => import('./Chart'));
<Suspense fallback={<p>Loading</p>}><Chart /></Suspense>
```

17. Forms at Scale

Use React Hook Form with Zod for validation, performant uncontrolled inputs and typed schemas.

```
const { register, handleSubmit } = useForm<FormData>({ resolver: zodResolver(schema) });
```

18. Server State with TanStack Query

Caching, refetching, pagination and mutations for API data with minimal code.

```
const { data, isLoading } = useQuery({ queryKey: ['users'], queryFn: fetchUsers });
```

19. Testing React

Test behaviour with Vitest/Jest and React Testing Library; mock APIs with MSW; end-to-end with Playwright.

```
render(<Counter />);
await userEvent.click(screen.getByRole('button'));
expect(screen.getByText('1')).toBeInTheDocument();
```

Master Level

20. React 19 Features

Actions, `useActionState`, `useOptimistic`, `use()` for promises, Server Components and the React Compiler reduce boilerplate and manual memoisation.

```
const [state, formAction] = useActionState(saveUser, null);
```

21. Project Architecture and Best Practices

Feature folders, absolute imports, error boundaries, accessibility, environment configs, linting and CI.

```
src/
  features/auth/
  features/products/
  components/ui/
  lib/
```

22. Real Project: E-commerce Front End

Product list, search, filters, cart (context/Zustand), checkout form, auth routes, TanStack Query for API, deployed on Vercel/Netlify.

23. Deployment and Interview Prep

Build with `npm run build`, deploy static files, set env vars, and practise explaining rendering, reconciliation and hooks rules.

```
npm run build
npx serve dist
```

Interview Question Bank

Q1. What is the virtual DOM?
An in-memory representation of the UI; React diffs it against the previous version and updates only the changed real DOM nodes.

Q2. State vs props?
Props are inputs passed from the parent (read-only); state is data owned and changed by the component.

Q3. Why are keys needed?
Keys let React identify list items between renders so it can update, move and remove them correctly.

Q4. Rules of hooks?
Call hooks only at the top level of components/custom hooks, never inside loops, conditions or nested functions.

Q5. useEffect dependency array?
Controls re-running: `[]` once, `[a]` when `a` changes, none after every render.

Q6. Controlled vs uncontrolled components?
Controlled inputs are driven by state; uncontrolled keep their value in the DOM, read via refs.

Q7. What is prop drilling and how to avoid it?
Passing props through many levels; use Context, composition or a state library.

Q8. `useMemo` vs `useCallback`?
`useMemo` memoises a computed value; `useCallback` memoises a function.

Q9. What is reconciliation?
The algorithm React uses to compare element trees and compute minimal updates.

Q10. What are Server Components?
Components rendered on the server that send no JavaScript to the client and can access data directly.

Capstone Project Ideas

1. Todo app with filters
2. Movie search app with TanStack Query
3. Kanban board with drag and drop
4. Blog with React Router and Markdown
5. E-commerce storefront